
Project Documentation

DHT sensor data monitoring

Augustin BRESSAC

Contents

1	Introduction	3
2	Developer manual	4
2.1	Git	4
2.2	Requirements	4
2.2.1	Backend (app.py)	4
2.2.2	Frontend (index.html)	4
2.2.3	Firmware (main.ino)	4
2.3	Architecture of the project	5
2.3.1	Firmware and data flow	5
2.3.2	Backend	5
2.3.3	Frontend	7
3	How to run the project	7
3.1	Connections	7
3.2	Running the project	8
4	User manual	8
4.1	Features	8
4.1.1	Running capture	8
4.1.2	Clear data	8
4.1.3	Set capture interval	9
4.1.4	Save log	9
4.1.5	Load log	9
4.1.6	Chart view	9
4.1.7	Gauge view	10
4.1.8	Table log view	11
4.1.9	Offline availability.	11
5	ThingsBoard comparaison (failed)	11
5.1	Work done	11
5.2	Incident	12

1 Introduction

The aim of this project is to monitor signal obtained from a real sensor thanks to a web application in order to fulfill the Internet of Things (IoT) concept.

This particular project will use a DHT sensor to monitor humidity and temperature of the environment.

The user will be able to interact with the system and to chose different types of visualizations (charts, gauges and table) of the data obtained by the sensor.

The system will also be able to work offline, even when the sensor is not connected.

Finnally, the solution should have been compared to the solution on the Things-Board platform, unfortunately, an incident occured (5).

2 Developer manual

2.1 Git

Here's the link to the git of the project :

Git Repository

Since I was working alone on the project and It was synchronized with my cloud, I did not make frequent commits to git.

2.2 Requirements

This project was done using Python 3.10.12 and probably works with all Python 3 versions.

2.2.1 Backend (app.py)

Flask : Framework for the API.

No library needed for database management because of sqlite being used and natively integrated with Python3.

Installation :

```
pip install Flask
```

2.2.2 Frontend (index.html)

Plotly : To display monitored or controlled data in the form of graphs in the client browser.

Gauge.js : To display monitored or controlled data in the form of gauges in the client browser

Installation :

no installation needed, files already in the repository.

2.2.3 Firmware (main.ino)

This project was done using arduinoIDE 2.3.8.

Libs :

ESP8266WiFi : manage wifi connection

DHT sensor library : read data from the sensor

Adafruit Unified Sensor : DHT lib dependency.

Installation :

For each library used, go to arduinoIDE > tools > manage libraries > search for the library and install it, the code using the library can now be uploaded to the nodeMCU.

2.3 Architecture of the project

2.3.1 Firmware and data flow

The firmware (main.ino) is a basic C++ code uploaded to the nodeMCU that get the data from the DHT sensor and send it through a http get request to the server (app.py), the server then manage the request and save the data into the database file data.db and the log file log.txt. Finally, the web browser display the data with the index.html page (cf 2.3.3).

Variables : There is 5 variables to change in this program to make it works :
DHTPIN : the nodeMUC pin connected to the “data” pin, I used GPIO 5 (cf 3.1).

ssid : ssid of the internet network.

password : password of the internet network.

serverIP : IP adress of the machine that will run the server.

serverPort : port chosen when creating the server, 5000 here.

2.3.2 Backend

Functions :

`init_db()`

Create a new sqlite database to store the data (and actions done by the user), here is the schema of the database :

```
CREATE TABLE IF NOT EXISTS sensor_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT,
    temperature REAL,
    humidity REAL,
    action TEXT)
```

temperature, humidity and action can be NULL since when there is an action there is no data and vice versa.

timestamp could have the “NOT NULL” mention, but I chose not to use it, because of potential failure of receiving it.

`save_to_db(temp, hum, action=“”)`

Store data and actions done by the user to the sqlite database.

When there is data, there is no action, when there is action there’s no data.

```
log_to_file(temp, hum, action="")
```

Write data and actions done by the user to the log file.

Same as the database, when there is data, there is no action, when there is action there's no data.

Routes :

/

render index.html

/update

GET, receive data and save it in db/file.

/api/status

GET, get the state of the system (open/close, monitoring or not), the route is used to display in the html the current state of the system

/api/parameters

POST, update the data gathering interval (in seconds), since it is action done by the user, save it in db/file.

/api/control

POST, manage the state of the system (open/close, monitoring or not), since it is action done by the user, save it in db/file.

/api/clear

POST, clear the database and the log file, it is action done by the user, but there is no point in storing it since we actually want to clear the log/db.

/api/save_log

POST, save the log in another text file that is stored in server/data/saves/.

/api/load_log

POST, load a previous saved file, this is done by overwriting the current db and log file with the content of the saved file.

/api/data

GET, get the last 15 entries of the database to display it.

2.3.3 Frontend

The frontend display the data in the form of a graph using the plotly library and in the form of a gauge using the gauge.js library.

Each action trigger the corresponding route in the backend.

The frontend check for new data every 2 seconds :

```
setInterval(updateData, 2000);
setInterval(updateStatus, 2000);
```

updateData is a function in index.html that calls the /api/data route and update the display accordingly to the data sent by the server.

3 How to run the project

First, make sure that every library needed is installed and a way to upload the main.ino code (project made using arduino IDE) to the nodeMCU is available (see 2.2).

3.1 Connections

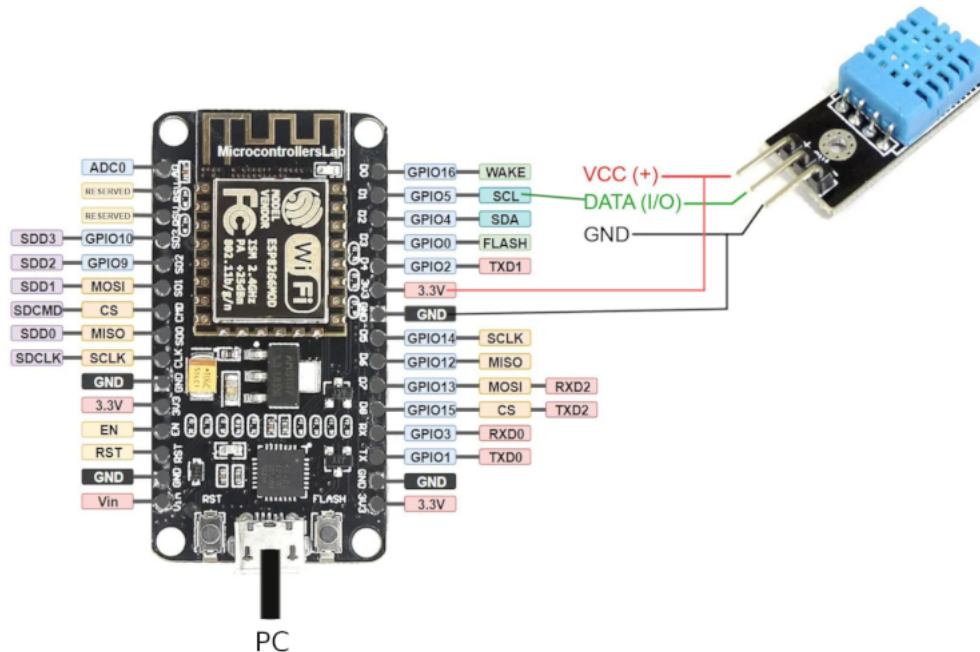


Figure 3.1: Pinout

VCC on any 3.3V pin.

GND on any GND pin.

DATA on any GPIO pin but change pin accordingly in the code if not GPIO5.

NodeMCU to the PC to upload the code.

3.2 Running the project

Upload main.ino to the nodeMCU.

Run the command (here from the root of the project)

```
sudo python3 server/app.py
```

Since the app is writing in/creating some files, sudo might be needed.

After running the command, open a web browser at the adress shown in the terminal or click [HERE](#).

4 User manual

Run the project following the instructions available in section 3.

4.1 Features

4.1.1 Running capture

After starting it, app will show the data currently in the database, including potential previous data.

To start a new capture (clear the existing data if you want), click on “open” to initialize the system (actually not really working like this, since the nodeMCU send data without knowing if anything receive it), then click on “start” to start the capture (wont work if system is closed).

Same as start/open, you can stop the capture and close the system with the corresponding buttons.

4.1.2 Clear data

Click on the “Clear data” button if you want to clear all the visualizations, the data will be lost if not saved with “Save Log” beforehand.

4.1.3 Set capture interval

Set the capture interval (in s) that you want, either using the arrows to increase/decrease it or by writing it.

It should work well with intervals $\geq 3s$, but since the nodeMCU takes some times to execute its capture loop, the actual interval might be longer for 1 and 2 seconds.

4.1.4 Save log

You can save the current data using the “Save Log” button, saved data will be stored in `~/server/data/saves` in the project repository.

4.1.5 Load log

You can load data from a log using the “Load Log” button, new data will overwrite current data so make sure to save it if you want.

4.1.6 Chart view

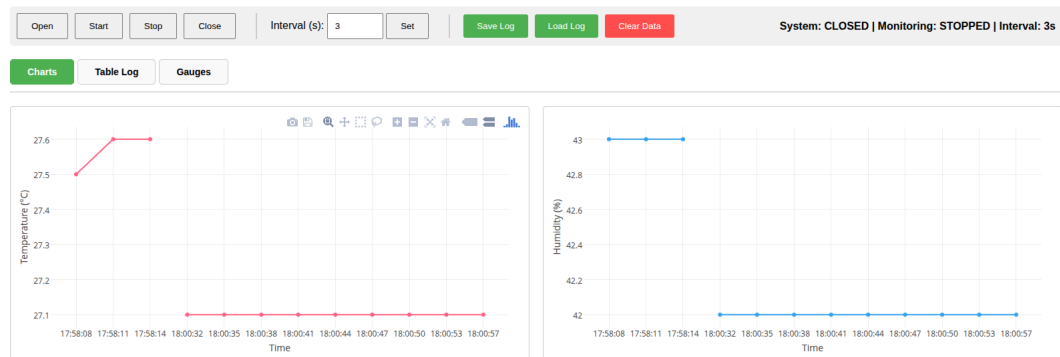


Figure 4.1: Chart view

The temperature and humidity are displayed on two separate charts, with time on the x axis.

The x axis is not scaled, discontinuity represent any kind of capture interruption (start/stop, offline, close app etc...).

Y axis scales automatically depending on the minimum and the maximum of captured data.

4.1.7 Gauge view

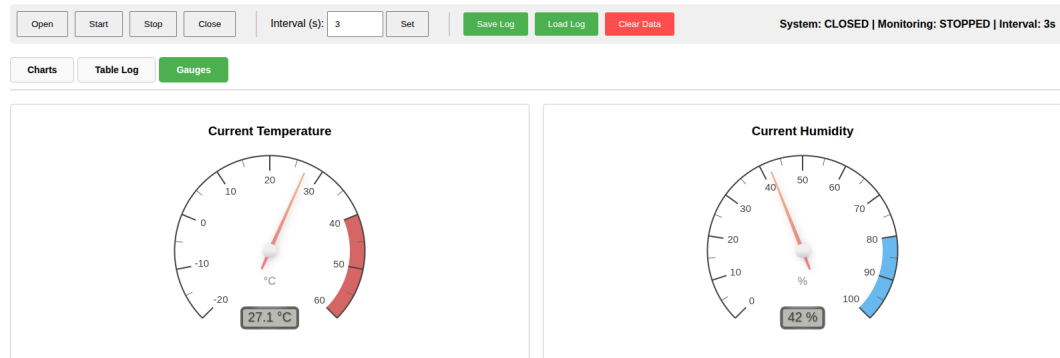


Figure 4.2: Gauge view

The “current” mention is a bit of a lie, it is actually the last recorded, so it is the current data only while the system is capturing it.

The edges of gauges are :

- -20 and 60°C for the temperature : common temperatures on earth with a margin.
- 0 to 100% for humidity since it is a percentage.

4.1.8 Table log view

Open

Start

Stop

Close

Interval (s):

3

Set

Save Log

Load Log

Clear Data

System: CLOSED | Monitoring: STOPPED | Interval: 3s

Charts

Table Log

Gauges

Data Log List

ID	Timestamp	Temperature (C)	Humidity (%)	Action
156	2026-06-03 18:00:57	27.1	42	
155	2026-06-03 18:00:53	27.1	42	
154	2026-06-03 18:00:50	27.1	42	
153	2026-06-03 18:00:47	27.1	42	
152	2026-06-03 18:00:45			SET INTERVAL=6
151	2026-06-03 18:00:44	27.1	42	
150	2026-06-03 18:00:41	27.1	42	
149	2026-06-03 18:00:38	27.1	42	
148	2026-06-03 18:00:35	27.1	42	
147	2026-06-03 18:00:32	27.1	42	
146	2026-06-03 18:00:31			START
145	2026-06-03 18:00:29			OPEN
144	2026-06-03 17:58:14	27.6	43	
143	2026-06-03 17:58:11	27.6	43	
142	2026-06-03 17:58:08	27.5	43	

Figure 4.3: Table log view

The table display only the 15 last rows but the saving and the load features are managing the whole data.

4.1.9 Offline availability.

Every features presented except the data capture is available even without the nodeMCU + sensor and/or internet connection.

5 ThingsBoard comparaison (failed)

5.1 Work done

I created an account on ThingsBoard, then a system in ThingsBoard and got the token of it that I should have used.

I wrote the code that was supposed to send data to ThingsBoard, using the instructions (medium.com) given in the assingment.

This code is still available in the git repository under the name thingsboard.ino in /server/firmware/.

5.2 Incident

When trying to upload this code on the nodeMCU, it crashed and even after replugging it, the usb port was not detected by arduinoIDE.

After I reseted the nodeMCU using the flash button, the usb port appeared again, but there was an error each times I tried to upload a code to the nodeMCU afterward.

The crash was apparently caused by a buffer overflow present in the code that I pasted from the instructions :

```
long rssi = WiFi.RSSI();  
char buffer[3];  
char* strptr = ltoa(rssi, buffer, 10);
```